CRYSTALS-KYBER: POST-QUANTUM CRYPTOGRAPHY

National Institute of Technology Karnataka

Sagar (252IS031), S.Pranai Govind (252IS036)

Impact of Quantum Computing

- A quantum computer could be used to quickly factor large integers, and also to solve the Discrete Logarithm problem efficiently, so this would have a drastic impact on public-key cryptography.
- The impact on secret-key cryptography would be much less severe.
- The main attack method on secret-key cryptography that could be carried out by a quantum computer is based on Grover's Algorithm.

Post-Quantum Cryptography (PQC)

- Quantum-safe cryptographic systems, also known as post-quantum cryptography, are designed to be resistant to attacks from both classical and quantum computers. These systems use algorithms that are believed to be secure even against quantum computers. Quantum-safe cryptography is becoming increasingly important as quantum computers continue to evolve and become more powerful.
- PQC has been a very active research area in the past few decades.
- Some actively researched PQC categories include
  - Lattice-based
  - Code-based
  - Multivariate
  - Hash/Symmetric key-based signatures
  - Elliptic curve isogeny-based

Why Post-Quantum Cryptography ?

Quantum Computers Can Break Traditional Cryptography :
Traditional cryptography like RSA and Diffie-Hellman is secure because classical computers cannot solve certain mathematical problems efficiently. However, quantum computers are much more powerful and can solve these problems quickly, which means they can break current security systems.

Shor's Algorithm Threatens Security :
Shor's Algorithm is a quantum algorithm that can efficiently factor large numbers and solve discrete logarithms. These problems are the foundation of RSA and Diffie-Hellman. If quantum computers run this algorithm, they can break these systems.

Need for Quantum-Resistant Alternatives :
Since quantum computers can break current cryptography, we need new systems that are secure even against quantum attacks. These are called post-quantum cryptographic systems. One important approach is lattice-based cryptography, such as Kyber, which is believed to be secure against quantum computers.

Selection Criteria By NIST

1. SECURE AGAINST BOTH CLASSICAL AND QUANTUM ATTACKS

Level	Security Description
I	At least as hard to break as AES128 (exhaustive key search)
II	At least as hard to break as SHA256 (collision search)
III	At least as hard to break as AES192 (exhaustive key search)
IV	At least as hard to break as SHA384 (collision search)
V	At least as hard to break as AES256 (exhaustive key search)

2. PERFORMANCE – Measured on various "classical" platforms

3. OTHER PROPERTIES

- Drop-in replacements – compatibility with existing protocols and networks
- Perfect forward secrecy
- Resistance to side-channel attacks
- Simplicity and flexibility
- Misuse resistance, etc.

The Selected Algorithms

- **ML-KEM (CRYSTALS-KYBER)**
  - KEM based on structured lattices
  - Good all-around performance and security
- **ML-DSA (CRYSTALS-DILITHIUM)**
  - Digital signature based on structured lattices
  - Good all-around performance and security, relatively simple implementation
  - NIST recommends it be the primary signature algorithm used
- **FN-DSA (FALCON)**
  - Digital signature based on structured lattices
  - Smaller bandwidth, but much more complicated implementation
  - The Falcon standard will come out after the others
- **SLH-DSA (SPHINCS+)**
  - Digital signature based on stateless hash-based cryptography
  - Solid security, but performance not as good compared to Dilithium/Falcon

What is CRYSTALS-Kyber?

CRYSTALS-Kyber (ML-KEM FIPS 203)

- CRYSTALS = Cryptographic Suite for Algebraic Lattices
- A **post-quantum cryptographic algorithm** selected by NIST
- Designed for **secure key exchange resistant to quantum attacks**

Background:

- NIST competition (2016–2022)
- 82 algorithms evaluated
- Kyber selected as **winner for Key Encapsulation (KEM)**

Standardization:

- Standard name: **ML-KEM (FIPS 203)**
- Published: **August 2024**

What is ML-KEM?

ML-KEM (Module-Lattice-based Key Encapsulation Mechanism)

- A **post-quantum Key Encapsulation Mechanism (KEM)**
- Based on **module lattice mathematics (MLWE problem)**
- Enables **secure shared key generation over public channels**
- Secure against **quantum computer attacks**

Key Idea:

- Two parties generate a **shared secret key** without directly sending it

Important Relation:

ML-KEM = Standardized version of CRYSTALS-Kyber

Modular arithmetic

- **Modulus:** $q \geq 2$.
- $a \equiv b \pmod{q}$ means that $a - b$ is an integer multiple of q.
- $r = a \bmod q$ means that r is the remainder upon dividing the integer a by q (so $0 \leq r \leq q - 1$).
- **Integers modulo q:**
$$\mathbb{Z}_q = \{0, 1, 2, \ldots, q - 1\}$$
where addition, subtraction and multiplication are performed modulo q.
- **Example:**
$$\mathbb{Z}_{17} = \{0, 1, 2, 3, \ldots, 16\}$$
  - In $\mathbb{Z}_{17}$:
$$9 + 15 = 7, \quad 9 - 15 = 11, \quad 9 \times 15 = 16$$
  - More precisely:
$$9 + 15 = 24 \equiv 7 \pmod{17}$$
$$9 - 15 = -6 \equiv 11 \pmod{17}$$
$$9 \times 15 = 135 \equiv 16 \pmod{17}$$

Polynomial rings

- Let q be a prime modulus.
- $\mathbb{Z}_q[x]$ is the set of all polynomials in x with coefficients in $\mathbb{Z}_q$.
- When adding, subtracting, multiplying and dividing polynomials in $\mathbb{Z}_q[x]$, all coefficient arithmetic is performed in $\mathbb{Z}_q$.
- **Example:** Let $q = 7$, and consider

$$f(x) = 5 + 4x^2 + 3x^3 \in \mathbb{Z}_7[x], \quad g(x) = 6 + 3x + 2x^2 \in \mathbb{Z}_7[x].$$

Then:

- $$f(x) + g(x) = 4 + 3x + 6x^2 + 3x^3$$
- $$f(x) - g(x) = 6 + 4x + 2x^2 + 3x^3$$
- $$f(x) \times g(x) = 2 + x + 6x^2 + 2x^3 + 3x^4 + 6x^5$$

The polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$

- Let q be a prime modulus, and let n be a positive integer.
- The polynomial ring
$$R_q = \mathbb{Z}_q[x]/(x^n + 1)$$
consists of all polynomials in $\mathbb{Z}_q[x]$ of degree less than n, where polynomial multiplication is performed modulo $(x^n + 1)$.
- So, to multiply polynomials $f(x), g(x) \in R_q$:
  1. Multiply $f(x)$ and $g(x)$ in $\mathbb{Z}_q[x]$ (with coefficients modulo q), obtaining a polynomial $h(x)$ of degree at most $2n - 2$.
  2. Divide $h(x)$ by $x^n + 1$ to get a remainder polynomial $r(x)$ of degree at most $n - 1$. (polynomials are reduced modulo $x^n + 1$.)
  3. Then $f(x) \times g(x) = r(x)$ in R_q.

  Key Idea: Reduce $h(x)$ modulo $(x^n + 1)$, i.e., replace $x^n \equiv -1 \bmod (x^n + 1)$, to get a remainder polynomial $r(x)$ of degree at most $n - 1$.
- **Note:** The size of R_q is q^n.

Example: The polynomial ring $R_q = \mathbb{Z}_{41}[x]/(x^4+1)$

- Let $q = 41$ and $n = 4$.
- Then R_q is comprised of the polynomials in $\mathbb{Z}_{41}[x]$ of degree at most 3.
- Let
$$f(x) = 32 + 17x^2 + 22x^3 \in R_q, \quad g(x) = 11 + 7x + 19x^2 + x^3 \in R_q.$$
- In $\mathbb{Z}_q[x]$,
$$h(x) = f(x) \times g(x) = 24 + 19x + 16x^2 + 24x^3 + 26x^4 + 25x^5 + 22x^6.$$
- The division of $h(x)$ by $x^4 + 1$ can be accomplished by replacing
$$x^4 \to -1, \quad x^5 \to -x, \quad x^6 \to -x^2,$$
and then simplifying.
- We obtain
$$\begin{aligned} r(x) &= 24 + 19x + 16x^2 + 24x^3 - 26 - 25x - 22x^2 \\ &= 39 + 35x + 35x^2 + 24x^3. \end{aligned}$$
- So,
$$f(x) \times g(x) = 39 + 35x + 35x^2 + 24x^3 \in R_q.$$

The module R_q^k

- Let k be a positive integer.
- R_q^k is the set of all vectors of length k, where each entry is a polynomial in R_q.
- **Addition** and **subtraction** of elements in R_q^k is component-wise (so the result is also an element in R_q^k).
- The **inner product** (multiplication) of two vectors in R_q^k results in a polynomial in R_q.
- All vectors in R_q^k will be written as column vectors.

Example: R_q^k

- Let $q = 137,\ n = 4,\ R_q = \mathbb{Z}_{137}[x]/(x^4+1),\ k = 3.$
- Let

$$a = \begin{pmatrix} 93 + 51x + 34x^2 + 54x^3 \\ 27 + 87x + 81x^2 + 6x^3 \\ 112 + 15x + 46x^2 + 122x^3 \end{pmatrix} \quad \text{and} \quad b = \begin{pmatrix} 40 + 78x + x^2 + 119x^3 \\ 11 + 31x + 57x^2 + 90x^3 \\ 108 + 72x + 47x^2 + 14x^3 \end{pmatrix} \in R_q^k$$

- Then

$$a + b = \begin{pmatrix} 133 + 129x + 35x^2 + 36x^3 \\ 38 + 118x + x^2 + 96x^3 \\ 83 + 87x + 93x^2 + 136x^3 \end{pmatrix} \qquad a - b = \begin{pmatrix} 53 + 110x + 33x^2 + 72x^3 \\ 16 + 56x + 24x^2 + 53x^3 \\ 4 + 80x + 136x^2 + 108x^3 \end{pmatrix}$$

- Inner product:

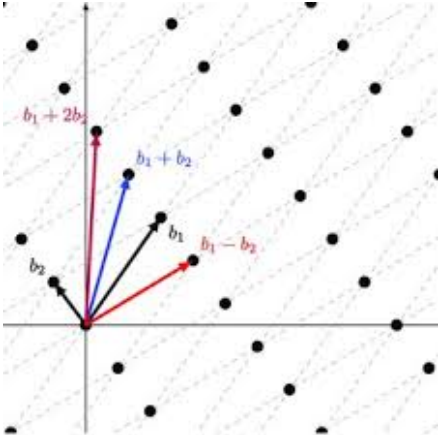
$$a^T b = a[1]b[1] + a[2]b[2] + a[3]b[3] = 93 + 59x + 44x^2 + 132x^3$$

Lattices

- A lattice is a set of points formed by integer combinations of basis vectors.

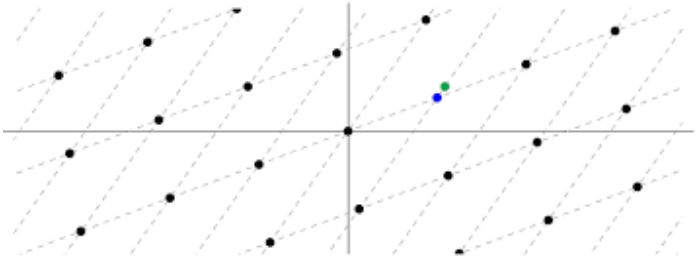
$$\mathcal{L} = \{z_1 b_1 + z_2 b_2 + \cdots + z_n b_n \mid z_i \in \mathbb{Z}\}$$

where $b_1, b_2, \ldots, b_n$ are basis vectors.



Lattices-Based Problem

- **Closest Vector Problem (CVP)**
  - Given a target point, find the closest lattice point.
  - These problems are believed to be hard to solve.
- Kyber is based on difficult lattice problems.
- In Kyber, noise shifts the original point, making recovery difficult.
- In high dimensions, solving this becomes computationally infeasible.



Blue Point: Original lattice point **Green Point:** Noisy target
Black Points: Valid lattice points

Challenge: Recover the original blue lattice point using only the noisy green point.(And that's incredibly hard)

MODULE LEARNING WITH ERRORS (MLWE)

- **Learning With Errors (LWE):**
  - Given a set of linear equations it is trivial to find a solution.
  - If you add 'noise' or 'error' to the equations it is very difficult to solve.
  - This is similar to lattice problems like Closest Vector Problem(CVP).

Without Noise (Easy)

$$A \cdot s = b$$

$$\begin{bmatrix} 4 & 1 & 11 & 10 \\ 5 & 5 & 9 & 5 \\ 3 & 9 & 0 & 10 \\ 1 & 3 & 3 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 9 \\ 11 \\ 11 \end{bmatrix} = \begin{bmatrix} 4 \\ 8 \\ 1 \\ 10 \end{bmatrix}$$

Easy!

With Noise (Hard)

$$A \cdot s + e = b$$

$$\begin{bmatrix} 4 & 1 & 11 & 10 \\ 5 & 5 & 9 & 5 \\ 3 & 9 & 0 & 10 \\ 1 & 3 & 3 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 9 \\ 11 \\ 11 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \\ 2 \\ 11 \end{bmatrix}$$

Hard!

MODULE LEARNING WITH ERRORS (MLWE)

- Instead of integer elements for the matrix and vectors, the elements are polynomials.
- The expression is written in matrix-vector form over the polynomial ring:

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

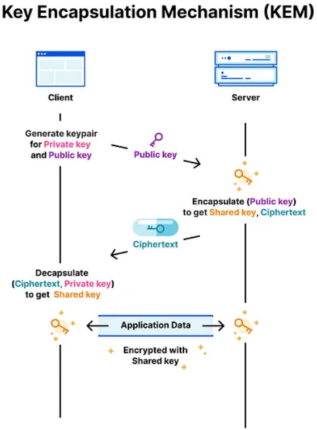
$$\begin{pmatrix} A_{0,0}(X) & \cdots & A_{0,k}(X) \\ \vdots & \ddots & \vdots \\ A_{k,0}(X) & \cdots & A_{k,k}(X) \end{pmatrix} \begin{pmatrix} s_0(X) \\ \vdots \\ s_k(X) \end{pmatrix} + \begin{pmatrix} e_0(X) \\ \vdots \\ e_k(X) \end{pmatrix} = \begin{pmatrix} t_0(X) \\ \vdots \\ t_k(X) \end{pmatrix}$$

$$As + e = t$$

- $A \in R_q^{k \times k}$: matrix of polynomials
- $s \in R_q^k$: secret polynomial vector
- $e \in R_q^k$: error polynomial vector

KYBER KEM FLOW

- Kyber uses a **Key Encapsulation Mechanism (KEM)** to securely establish a shared secret key.
- The client generates a public/private key pair.
- The server uses the public key to generate:
  - Ciphertext
  - Shared secret
- The client uses the ciphertext and private key to recover the same shared secret.



KYBER KEM FLOW: STEPS

1. **Key Generation** (client/ Receiver side)
   - Generate public key and private key
   - Send public key to server
2. **Encapsulation** (server/ sender side)
   - Use client public key to generate:
     - Ciphertext
     - Shared secret
   - Send ciphertext to client
3. **Decapsulation** (client/ Receiver side)
   - Use ciphertext + private key
   - Recover same shared secret
4. **Secure Communication**
   - Both parties use the shared key to encrypt application data securely.

KYBER: KEM MATHEMATICAL FLOW

Roles

- **Alice (Receiver):** generates key pair and receives ciphertext
- **Bob (Sender):** uses Alice's public key to send shared secret

1. Key Generation (Alice)

$$R_q = \mathbb{Z}_q[X]/(X^n + 1)$$

Alice samples:

$$\hat{A} \in R_q^{k \times k}, \quad s_0, e_0 \in R_q^k$$

Computes:

$$p_0 = \hat{A} s_0 + e_0$$

- Public key: $(\hat{A}, p_0)$
- Private key: s_0

KYBER: KEM MATHEMATICAL FLOW

2. Encapsulation (Bob)

Bob uses Alice's public key:

$$(\hat{A}, p_0)$$

Bob chooses random:

$$m,\ e \in R_q, \qquad s_1,\ e_1 \in R_q^k$$

Bob computes:

$$p_1 = s_1^T \hat{A} + e_1^T$$

$$c = \left\lfloor \frac{q}{2} \right\rfloor m + e + s_1^T p_0$$

Ciphertext sent:

$$CT = (c, p_1)$$

KYBER: KEM MATHEMATICAL FLOW

3. Decapsulation

Alice receives:

$$(c, p_1), \quad s_0$$

Alice computes:

$$c - p_1 s_0$$

Using:

$$p_0 = \hat{A} s_0 + e_0, \qquad p_1 = s_1^T \hat{A} + e_1^T$$

This simplifies to:

$$c - p_1 s_0 = \left\lfloor \frac{q}{2} \right\rfloor m + e + s_1^T e_0 - e_1^T s_0$$

$$c - p_1 s_0 = \left\lfloor \frac{q}{2} \right\rfloor m + \text{small noise}$$

- After decapsulation, Alice has the encoded message plus only a small amount of noise. Alice can recover the original message by decoding.

KYBER: KEM MATHEMATICAL FLOW

4. Message Recovery

Alice decodes:

- Near $0 \to$ bit 0
- Near $\frac{q}{2} \to$ bit 1

Thus:

$$m$$

5. Shared Secret Generation

Both compute:

$$K = \text{Hash}(m)$$

- Bob obtains K
- Alice obtains K

Secure shared secret established.

EXAMPLE: KYBER ENCAPSULATION

Small MLWE Example to Illustrate Kyber

Polynomial Ring:

$$R_q = \mathbb{Z}_{17}[x]/(x^4 + 1)$$

Alice's Key Generation

$$\hat{A} = \begin{pmatrix} 6x^3 + 16x^2 + 16x + 11 & 9x^3 + 4x^2 + 6x + 3 \\ 5x^3 + 3x^2 + 10x + 1 & 6x^3 + x^2 + 9x + 15 \end{pmatrix}$$

$$s_0 = \begin{pmatrix} -x^3 - x^2 + x \\ -x^3 - x \end{pmatrix}, \quad e_0 = \begin{pmatrix} x^2 \\ x^2 - x \end{pmatrix}$$

EXAMPLE: KEY GENERATION

Alice Computes:

$$p_0 = \hat{A}s_0 + e_0$$

$$p_0 = \begin{pmatrix} 16x^3 + 15x^2 + 7 \\ 10x^3 + 12x^2 + 11x + 6 \end{pmatrix}$$

$$\text{Public key } = (\hat{A}, p_0), \quad \text{Private key } = s_0$$

Alice sends public key to bob :

$$(\hat{A}, p_0)$$

EXAMPLE: ENCAPSULATION

Bob chooses random:

$$s_1 = \begin{pmatrix} -x^3 + x^2 \\ x^3 + x^2 - 1 \end{pmatrix}, \quad e_1 = \begin{pmatrix} x^2 + x \\ x^2 \end{pmatrix}, \quad e = -x^3 - x^2$$

Message:

$$11_{10} = 1011_2$$

$$m = x^3 + x + 1$$

Scaled:

$$\left\lfloor \frac{q}{2} \right\rfloor m = 9x^3 + 9x + 9$$

Bob computes:

$$p_1 = s_1^T \hat{A} + e_1^T$$

$$c = \left\lfloor \frac{q}{2} \right\rfloor m + e + s_1^T p_0$$

EXAMPLE: DECAPSULATION (STEP 1)

Alice receives ciphertext:

$$CT = (c, p_1)$$

Where:

$$p_1 = \begin{pmatrix} 11x^3 + 11x^2 + 10x + 3 \\ 4x^3 + 4x^2 + 13x + 11 \end{pmatrix}$$

$$c = 7x^3 + 6x^2 + 8x + 15$$

Alice computes:

$$m_n = c - p_1^T s_0$$

Substituting values:

$$m_n = 7x^3 + 14x^2 + 7x + 5$$

Observation:
Recovered polynomial contains message + small noise.

EXAMPLE: DECAPSULATION (STEP 2)

Recovered noisy polynomial:

$$m_n = 7x^3 + 14x^2 + 7x + 5$$

Rounding Rule: $\text{dist}(x, a) = \min(|x - a|,\ q - |x - a|)$ Each coefficient is rounded to the nearest of $\{0,\ \lfloor q/2 \rfloor\}$.

Round coefficients to nearest message levels: $7 \to 9, \qquad 14 \to 0, \qquad 7 \to 9, \qquad 5 \to 9$
Rounded polynomial:

$$m_n = 9x^3 + 0x^2 + 9x + 9$$

Scale down:

$$m_n = \left\lfloor \frac{q}{2} \right\rfloor m_b$$

$$m_b = \frac{1}{9}(9x^3 + 0x^2 + 9x + 9) = x^3 + x + 1$$

EXAMPLE: FINAL MESSAGE RECOVERY

Recovered message polynomial:

$$m_b = x^3 + x + 1$$

Convert polynomial back to binary:

$$x^3 + x + 1 \ \rightarrow \ 1x^3 + 0x^2 + 1x + 1 \rightarrow \ 1011_2$$

Original message recovered:

$$1011_2 \rightarrow 11_{10}$$

Shared Secret Derivation:

Now Recovered message is used to derive the shared secret key.

$$K = \text{Hash}(m)$$

Both Alice and Bob derive the same shared secret.

Kyber Variants

Different Kyber variants provide varying levels of security, allowing for flexibility in implementation.

Kyber512

Security Level equivalent to **AES-128**

Kyber768

Security Level equivalent to **AES-192**

Kyber1024

Security Level equivalent to **AES-256**

Kyber Analysis

Algorithm	Public Key	Ciphertext	Secret Key	Security Level	Equivalent
RSA-3072	384 B	384 B	384 B	Broken (Quantum)	AES128
ECC (256)	64 B	64 B	32 B	Broken (Quantum)	AES128
Kyber-512	800 B	768 B	1632 B	Quantum-Safe	AES128
Kyber-768	1184 B	1088 B	2400 B	Higher Security	AES192
Kyber-1024	1568 B	1568 B	3168 B	Extremely Secure	AES256

Advantages & Applications

Advantages of Kyber

Kyber offers strong quantum resistance, ensuring security against future quantum attacks. It provides efficient performance and fast computation, making it suitable for real-time applications. Being NIST-approved enhances its reliability and trustworthiness for secure communications.

Applications of Kyber

Kyber is widely used in secure communication protocols such as TLS and HTTPS. It is applied in VPNs for private networks, as well as in cloud and IoT security to protect data transmission and storage. Its versatility makes it important in modern cybersecurity systems.

Conclusion

- CRYSTALS-Kyber is a **post-quantum key exchange algorithm**
- It is based on **lattice cryptography**, which is hard to break even for quantum computers
- Standardized as **ML-KEM (FIPS 203)** by NIST
- Plays a key role in **future secure communication systems**

In short:

Kyber ensures security in the era of quantum computing.

References

- NIST, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, 2024.
  `https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf`
- CRYSTALS-Kyber Official Website:
  `https://pq-crystals.org/kyber/`
- Cryptography101, *Kyber and Dilithium Explained*:
  `https://cryptography101.ca/kyber-dilithium/`
- Medium Article, *CRYSTALS-Kyber: The Key to Post-Quantum Encryption*:
  `https://medium.com/identity-beyond-borders/crystals-kyber-the-key-to-post-quantum-encryption-3154b305e7bd`
- YouTube Explanation Video:
  `https://youtu.be/evAx8_q5wMw`

Thank You!

Questions?

CRYSTALS-Kyber (ML-KEM FIPS 203)
Securing the Future of Cryptography

Presented by: Sagar & Pranai Govind